

# Optimization (cont.)

## Lecture 25

Dr. Colin Rundel

# Regression via `optim()`

# Simulated data

```
1 set.seed(42)
2 n = 200
3 d = tibble(
4   x1 = rnorm(n),
5   x2 = rnorm(n),
6   x3 = rnorm(n),
7   x4 = rnorm(n),
8   x5 = rnorm(n),
9   y = 3 + 2*x1 - 1.5*x2 + 0*x3 + 0.1*x4 + 0*x5 + rnorm(n)
10 )
```

True coefficients:  $\beta_0 = 3, \beta_1 = 2, \beta_2 = -1.5, \beta_3 = 0, \beta_4 = 0.1, \beta_5 = 0$

# OLS as optimization

The least squares objective is

$$\text{RSS}(\beta) = \sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

```
1 X = model.matrix(y ~ ., data = d)
2
3 rss = function(beta) {
4   sum((d$y - X %*% beta)^2)
5 }
```

# Fitting with `optim()`

```
1 res_ols = optim(  
2   rep(0, 6), rss, method = "BFGS"  
3 )  
4 res_ols
```

```
$par  
[1] 2.96159673 2.01899624 -1.56332824 0.01780  
[6] -0.08863705
```

```
$value  
[1] 194.6853
```

```
$counts  
function gradient  
      43      10
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

```
1 coef(lm(y ~ ., data = d))
```

```
(Intercept)          x1          x2          x3  
2.96159674  2.01899624 -1.56332824  0.01780703  
          x5  
-0.08863705
```

# Lasso regression

The Lasso adds an  $L_1$  penalty to shrink coefficients toward zero (note: we don't penalize the intercept),

$$\text{RSS}(\beta) + \lambda \sum_{j=1}^p |\beta_j|$$

```
1 lasso_obj = function(beta, lambda) {  
2   sum((d$y - X %*% beta)^2) + lambda * sum(abs(beta[-1]))  
3 }
```

# Lasso results

Since the  $L_1$  penalty is not differentiable, we use Nelder-Mead.

```
1 fit_lasso = function(lambda) {  
2   res = optim(rep(0, 6), lasso_obj, lambda = lambda, method = "Nelder-Mead",  
3             control = list(maxit = 10000))  
4   setNames(res$par, colnames(X))  
5 }
```

```
1 rbind(  
2   "OLS"           = coef(lm(y ~ ., data = d)),  
3   "Lasso (λ=10)"  = fit_lasso(10),  
4   "Lasso (λ=50)"  = fit_lasso(50),  
5   "Lasso (λ=100)" = fit_lasso(100),  
6   "Truth"         = c(3, 2, -1.5, 0, 0.1, 0)  
7 ) |> round(3)
```

|               | (Intercept) | x1    | x2     | x3     | x4     | x5     |
|---------------|-------------|-------|--------|--------|--------|--------|
| OLS           | 2.962       | 2.019 | -1.563 | 0.018  | 0.080  | -0.089 |
| Lasso (λ=10)  | 2.958       | 1.988 | -1.533 | 0.000  | 0.048  | -0.060 |
| Lasso (λ=50)  | 2.910       | 1.427 | -1.519 | -0.058 | -0.342 | 0.183  |
| Lasso (λ=100) | 3.085       | 1.495 | -1.476 | -0.001 | 0.203  | 0.000  |
| Truth         | 3.000       | 2.000 | -1.500 | 0.000  | 0.100  | 0.000  |

# Maximum Likelihood Estimation



# The idea

Given observed data  $x_1, \dots, x_n$  and a parametric model  $f(x|\theta)$ , the **likelihood** is

$$\square(\theta|\mathbf{x}) = \prod_{i=1}^n f(x_i|\theta)$$

The **log-likelihood** is easier to work with,

$$\ell(\theta|\mathbf{x}) = \sum_{i=1}^n \log f(x_i|\theta)$$

The **MLE** is the parameter value that maximizes the likelihood (equivalently, minimizes the negative log-likelihood),

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \ell(\theta|\mathbf{x}) = \arg \min_{\theta} -\ell(\theta|\mathbf{x})$$

# Normal MLE

```
1 set.seed(1234)
2 x = rnorm(100, mean = -3.2, sd = 1.25)
3 c(mean = mean(x), sd = sd(x))
```

| mean      | sd       |
|-----------|----------|
| -3.395952 | 1.255507 |

```
1 nll_norm = function(theta) {
2   -sum(dnorm(x, mean = theta[1],
3             sd = theta[2], log = TRUE))
4 }
```

```
1 nll_norm(c(0, 1))
```

```
[1] 746.5451
```

```
1 nll_norm(c(-3, 1))
```

```
[1] 177.7595
```

```
1 nll_norm(c(-3.2, 1.25))
```

```
[1] 165.374
```

# Fitting the Normal MLE

```
1 optim(c(0, 1), nll_norm, method = "BFGS")
```

```
$par  
[1] -56.40254 202.27205
```

```
$value  
[1] 626.2908
```

```
$counts  
function gradient  
      101      100
```

```
$convergence  
[1] 1
```

```
$message  
NULL
```

Note: `sd` must be positive, but BFGS doesn't know that — we got lucky with our starting value. We'll address this shortly.

# Gamma MLE

```
1 set.seed(5678)
2 x_gam = rgamma(200, shape = 3, rate = 2)
3 c(mean = mean(x_gam), var = var(x_gam))
```

```
      mean      var
1.4527658 0.8509466
```

```
1 nll_gamma = function(theta) {
2   -sum(dgamma(x_gam, shape = theta[1],
3             rate = theta[2], log = TRUE))
4 }
```

```
1 optim(
2   c(1, 1), nll_gamma, method = "BFGS"
3 )
```

```
$par
[1] 2.924804 2.013266
```

```
$value
[1] 226.3538
```

```
$counts
function gradient
      28      10
```

```
$convergence
[1] 0
```

```
$message
NULL
```

# Beta MLE

```
1 set.seed(9012)
2 x_beta = rbeta(200, shape1 = 2, shape2 = 5)
3 c(mean = mean(x_beta), var = var(x_beta))
```

```
      mean      var
0.30985082 0.02450097
```

```
1 nll_beta = function(theta) {
2   -sum(dbeta(x_beta, shape1 = theta[1],
3             shape2 = theta[2], log = TRUE))
4 }
```

```
1 optim(
2   c(1, 1), nll_beta, method = "BFGS"
3 )
```

```
$par
[1] 2.484288 5.506032
```

```
$value
[1] -99.01453
```

```
$counts
function gradient
      40      13
```

```
$convergence
[1] 0
```

```
$message
NULL
```

# Constrained Parameters

# The problem

Many distribution parameters have restricted support — e.g.,  $\sigma > 0$ ,  $\text{shape} > 0$ ,  $\text{rate} > 0$ .

If `optim()` tries a negative value, the log-likelihood returns `NaN`:

```
1 dnorm(0, mean = 0, sd = -1, log = TRUE)
```

```
[1] NaN
```

```
1 dgamma(1, shape = -1, rate = 1, log = TRUE)
```

```
[1] NaN
```

This causes the optimizer to fail or give unreliable results. Two common solutions:

1. **Reparameterization** — optimize over a transformed, unconstrained space
2. **Box constraints** — use `L-BFGS-B` with `lower` / `upper` bounds

# Reparameterization

Optimize over  $\log(\sigma)$  instead of  $\sigma$ , using `exp()` to map back to the positive reals.

```
1 nll_norm_reparam = function(theta) {  
2   -sum(dnorm(  
3     x, mean = theta[1],  
4     sd = exp(theta[2]), log = TRUE  
5   ))  
6 }
```

```
1 res = optim(  
2   c(0, log(1)), nll_norm_reparam,  
3   method = "BFGS"  
4 )  
5 res
```

```
$par  
[1] -3.3959601  0.2225083
```

```
$value  
[1] 164.1453
```

```
$counts  
function gradient  
      27      11
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

```
1 c(mu = res$par[1], sigma = exp(res$par[2]))
```

```
      mu      sigma  
-3.395960 1.249206
```



# Reparameterized Gamma MLE

Both shape and rate must be positive — optimize over their logs.

```
1 nll_gamma_reparam = function(theta) {  
2   -sum(dgamma(  
3     x_gam,  
4     shape = exp(theta[1]),  
5     rate = exp(theta[2]),  
6     log = TRUE  
7   ))  
8 }
```

```
1 res_gam = optim(  
2   c(0, 0), nll_gamma_reparam,  
3   method = "BFGS"  
4 )  
5 exp(res_gam$par)
```

```
[1] 2.924794 2.013259
```

# Box constraints with L-BFGS-B

An alternative — specify bounds directly via `lower` and `upper` (only available for "L-BFGS-B").

```
1 optim(  
2   c(0, 1), nll_norm,  
3   method = "L-BFGS-B",  
4   lower = c(-Inf, 1e-6),  
5   upper = c(Inf, Inf)  
6 )
```

```
$par  
[1] -3.395950  1.249205
```

```
$value  
[1] 164.1453
```

```
$counts  
function gradient  
      17      17
```

```
$convergence  
[1] 0
```

```
$message  
[1] "CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EP
```

```
1 optim(  
2   c(1, 1), nll_gamma,  
3   method = "L-BFGS-B",  
4   lower = c(1e-6, 1e-6),  
5   upper = c(Inf, Inf)  
6 )
```

```
$par  
[1] 2.924803 2.013266
```

```
$value  
[1] 226.3538
```

```
$counts  
function gradient  
      11      11
```

```
$convergence  
[1] 0
```

```
$message  
[1] "CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EP
```

# Reparameterization vs. bounds

|               | Reparameterization           | Box Constraints         |
|---------------|------------------------------|-------------------------|
| Methods       | Any (BFGS, CG, ...)          | L-BFGS-B only           |
| Gradients     | Need chain rule adjustment   | Work on original scale  |
| Flexibility   | Any smooth transform         | Lower/upper bounds only |
| Edge behavior | Can't reach boundary exactly | Can sit on boundary     |

In general, **reparameterization is more flexible** and works with any method. Use **bounds** when you want simplicity and L-BFGS-B is sufficient.

# Automatic Differentiation in Base R

# The gradient problem

Throughout this lecture we've been supplying hand-coded gradient functions to `optim()`. This works but is:

- **Error-prone** — easy to make mistakes in the calculus or the code
- **Tedious** — especially for complex objective functions
- **Hard to maintain** — changing the objective means updating the gradient too

Base R provides tools for **symbolic differentiation** that can compute derivatives automatically: `D()`, `deriv()`, and `deriv3()`.

# D() — symbolic derivatives

D() differentiates an R **expression** with respect to a named variable and returns a new expression.

```
1 D(expression(x^2), "x")
```

2 \* x

```
1 D(expression(sin(x) * cos(y)), "x")
```

cos(x) \* cos(y)

```
1 D(expression(sin(x) * cos(y)), "y")
```

-(sin(x) \* sin(y))

```
1 D(expression(x^2 + 3*x*y + y^2), "x")
```

2 \* x + 3 \* y

```
1 D(expression(x^2 + 3*x*y + y^2), "y")
```

3 \* x + 2 \* y

# Using `D()` to build gradient functions

We can evaluate the result of `D()` by providing variable values,

```
1 df_dx = D(expression(x^4 + x^3 - x^2 - x), "x")
2 df_dx
```

$4 * x^3 + 3 * x^2 - 2 * x - 1$

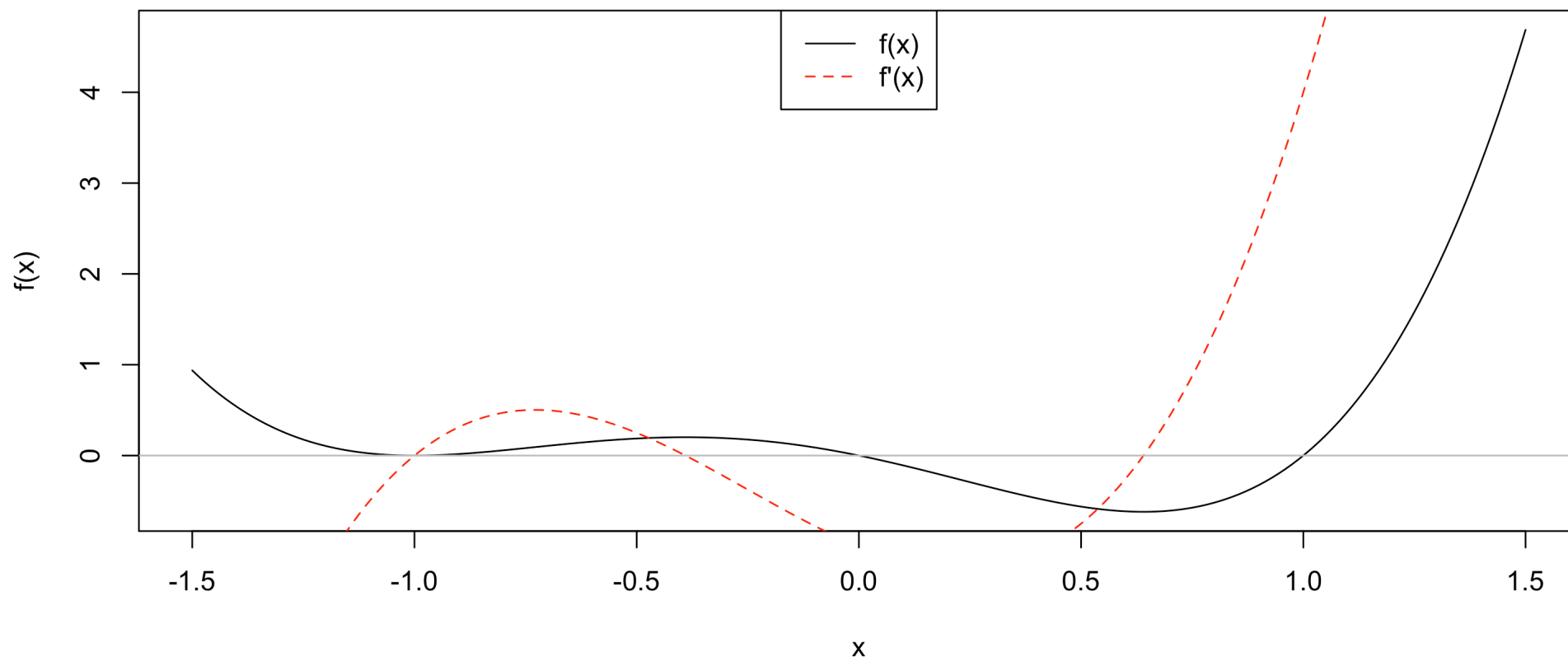
```
1 eval(df_dx, list(x = 0))
```

[1] -1

```
1 eval(df_dx, list(x = 1))
```

[1] 4

```
1 x = seq(-1.5, 1.5, length.out = 201)
2 plot(x, x^4 + x^3 - x^2 - x, type = "l", ylab = "f(x)")
3 lines(x, eval(df_dx, list(x = x)), col = "red", lty = 2)
4 abline(h = 0, col = "grey")
5 legend("top", c("f(x)", "f'(x)"), col = c("black", "red"), lty = c(1, 2))
```





# Higher-order and chained derivatives

`D()` can be applied repeatedly for higher-order derivatives,

```
1 expr = expression(x^4 + x^3 - x^2 - x)
2 D(expr, "x")
```

$$4 * x^3 + 3 * x^2 - 2 * x - 1$$

```
1 D(D(expr, "x"), "x")
```

$$4 * (3 * x^2) + 3 * (2 * x) - 2$$

```
1 D(D(D(expr, "x"), "x"), "x")
```

$$4 * (3 * (2 * x)) + 3 * 2$$

## D() limitations

D() only differentiates with respect to **one variable at a time** and returns an expression (not a function).

For multivariate functions where we want a callable function that returns both the value *and* gradient, we need `deriv()`.

# deriv() — gradient functions

`deriv()` takes a formula or expression and a character vector of variable names, and returns a function that computes the value and its "gradient" attribute.

```
1 f_grad = deriv(  
2   ~ x^2 + 3*x*y + y^2,  
3   namevec = c("x", "y"),  
4   function.arg = TRUE  
5 )  
6 f_grad
```

```
function (x, y)  
{  
  .expr2 <- 3 * x  
  .value <- x^2 + .expr2 * y + y^2  
  .grad <- array(0, c(length(.value), 2L), list(NULL, c("x",  
    "y")))  
  .grad[, "x"] <- 2 * x + 3 * y  
  .grad[, "y"] <- .expr2 + 2 * y  
  attr(.value, "gradient") <- .grad  
  .value  
}
```

```
1 f_grad(x = 1, y = 2)
```

```
[1] 11  
attr(,"gradient")  
      x y  
[1,] 8 7
```

# Extracting the gradient

```
1 res = f_grad(x = 1, y = 2)
```

```
1 drop(res)
```

```
[1] 11  
attr(,"gradient")  
      x y  
[1,] 8 7
```

```
1 attr(res, "gradient")
```

```
      x y  
[1,] 8 7
```

The gradient at  $(1, 2)$  is  $(2(1) + 3(2), 3(1) + 2(2)) = (8, 7)$  — matching the analytical result.

# deriv3() — gradient *and* Hessian

`deriv3()` works exactly like `deriv()` but also computes the Hessian matrix (second derivatives), stored in a `"hessian"` attribute.

```
1 f_all = deriv3(  
2   ~ x^2 + 3*x*y + y^2,  
3   namevec = c("x", "y"),  
4   function.arg = TRUE  
5 )  
6 res = f_all(x = 1, y = 2)  
7 drop(res)
```

```
[1] 11  
attr(,"gradient")  
      x y  
[1,] 8 7  
attr(,"hessian")  
      , , x  
  
      x y  
[1,] 2 3  
  
      , , y
```

```
      x y  
[1,] 3 2
```

```
1 attr(,"gradient")
```

```
      x y  
[1,] 8 7
```

```
1 attr(,"hessian")
```

```
, , x
```

```
      x y  
[1,] 2 3
```

```
, , y
```

```
      x y  
[1,] 3 2
```

# Using `deriv()` with `optim()`

We can use `deriv()` to automatically supply gradients to `optim()`,

```
1 rosenbrock = deriv(  
2   ~ (1 - x)^2 + 100 * (y - x^2)^2,  
3   namevec = c("x", "y"),  
4   function.arg = TRUE  
5 )
```

```
1 fn = function(par) {  
2   res = rosenbrock(x = par[1], y = par[2])  
3   drop(res)  
4 }  
5 gr = function(par) {  
6   res = rosenbrock(x = par[1], y = par[2])  
7   drop(attr(res, "gradient"))  
8 }  
9 optim(c(-1, 1), fn, gr, method = "BFGS")
```

```
$par  
[1] 1 1
```

```
$value  
[1] 5.769286e-17
```

```
$counts
```



```
function gradient  
      117      51
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

# What `D()` / `deriv()` support

These functions handle a specific set of R math operations:

| Supported    | Examples                                                                                                                                                                                                                                                     |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Arithmetic   | <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>^</code>                                                                                                                                                                           |
| Trig         | <code>sin</code> , <code>cos</code> , <code>tan</code>                                                                                                                                                                                                       |
| Logarithms   | <code>log</code> , <code>exp</code> , <code>log2</code> , <code>log10</code>                                                                                                                                                                                 |
| Other math   | <code>sqrt</code> , <code>abs</code> , <code>gamma</code> , <code>lgamma</code> , <code>beta</code> , <code>lbeta</code> , <code>digamma</code> , <code>trigamma</code>                                                                                      |
| Distribution | <code>dnorm</code> , <code>pnorm</code> , <code>dbinom</code> , <code>dpois</code> , <code>dgamma</code> , <code>dbeta</code> , <code>dt</code> , <code>df</code> , <code>dchisq</code> , <code>dweibull</code> , <code>dlnorm</code> , <code>dcauchy</code> |

Not supported: control flow (`if`, `for`), matrix operations, or arbitrary R functions — for those you need numerical differentiation (e.g. `numDeriv::grad()`) or AD packages.